

Projetos em Ambientes Web e Cloud

Licenciatura em Tecnologias Digitais e
Inteligência Artificial

Aula 4

João Matos

joao_miguel_goncalves_matos@iscte-iul.pt

2024 – 2025

2º Semestre

Exercício

- box-shadow
- Google Font (para o título)
- background-image
- Flexbox containers
- transition: change background color of button on hover

#e97132ff

FOR SALE

#757575

Consultar



Exercício



Vila in Sesimbra

 Sesimbra, Portugal

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor.



Price
\$1000

CONSULTAR

```
<div class="house_card">
  <div class="house_sale"><p>For Sale</p></div>
  <div class="house_img"></div>
  <div class="house_title">
    <h2>Vila in Sesimbra</h2>
    <div class="location">
      
      <p>Sesimbra, Portugal</p>
    </div>
    <p>Lorem ipsum dolor sit amet, consectetur
      adipiscing elit, sed do eiusmod tempor.</p>
  </div>
  <div class="house_amenities">
    
    
    
    
  </div>
  <div class="house_button">
    <div class="price">
      <h3>Price</h3>
      <p>$1000</p>
    </div>
    <button class="button">Consultar</button>
  </div>
</div>
```

Exercício



Vila in Sesimbra

 Sesimbra, Portugal

Lorem ipsum dolor sit amet,
consectetur adipiscing elit, sed do
eiusmod tempor.



Price
\$1000

CONSULTAR

```
/* Apartment Card */
.house_card {
  position: relative;
  font-family: monospace;
  height: auto;
  border-radius: 10px;
  overflow: hidden;
  box-shadow: 0px 10px 15px -3px rgba(0,0,0,0.1);
}
.house_img {
  height: 200px;
  background-image: url("../imagens/apartment.jpeg");
  background-size: cover;
  background-position: center;
}
.house_sale {
  position: absolute;
  background: #e97132ff;
  padding: 10px 20px;
  color: white;
  text-transform: uppercase;
  top: 20px;
  left: 25px;
}
.house_sale p {
  margin: 0;
}
.location {
  display: flex;
  align-items: center;
  margin: 10px 0;
}
.house_amenities {
  display: flex;
  justify-content: space-between;
  padding: 10px 10px 20px;
  margin: 15px 20px;
  border-bottom: solid 0.5px #757575;
}
.house_title {
  padding: 15px 10px;
}
.price {
  display: flex;
  flex-direction: column;
  width: 100px;
}
.price p,
.price h3 {
  margin: 0;
}
.house_button {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0 20px 20px;
}
.house_button a,
.house_button button {
  padding: 12px 20px;
  text-align: center;
  border: solid 1px black;
  text-decoration: none;
  color: black;
  text-transform: uppercase;
  background-color: #ffffff;
}
```

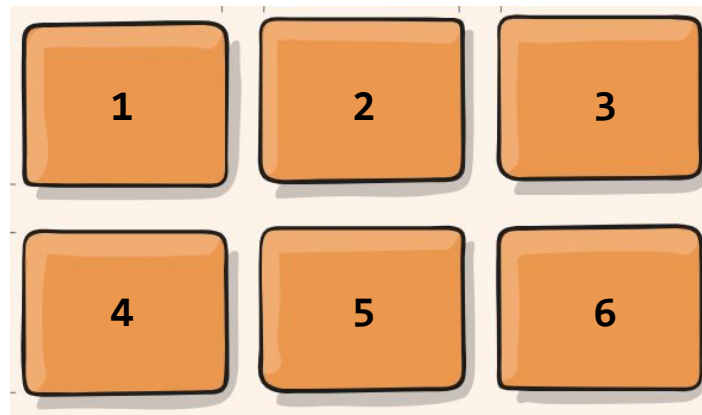
Display: grid

Usado para criar *layouts* modernos e flexíveis.

Sistema de layout baseado em **grelha**: com linhas e colunas facilitando o design de páginas web

```
<div class="grid-container">
  <div class="item1">1</div>
  <div class="item2">2</div>
  <div class="item3">3</div>
  <div class="item4">4</div>
  <div class="item5">5</div>
  <div class="item6">6</div>
</div>
```

```
.grid-container {
  display: grid;
}
```



Display: grid

Definimos a propriedade display com o valor grid no elemento *parent*

Os elementos diretos (*child*) tornam-se automaticamente *itens grid*

```
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto;
  background-color: #2196F3;
  padding: 10px;
}
```

1	2	3
4	5	6

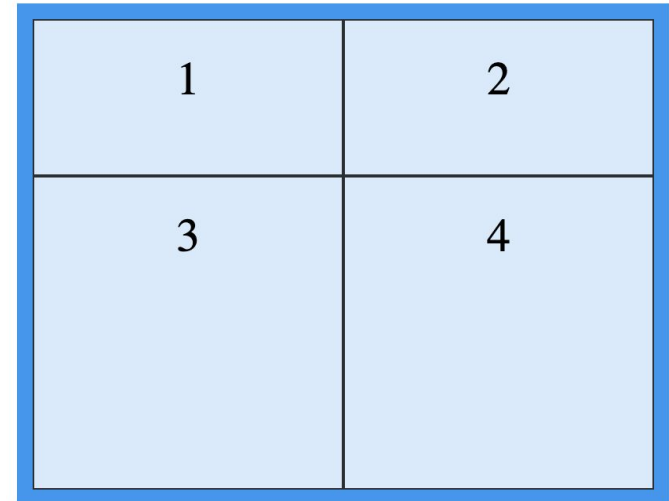
Display: grid

Definir linhas e colunas

grid-template-columns ; **grid-template-rows**

Ao definir o elemento *parent* como um grid, podemos especificar quantas colunas e linhas ele terá (e os tamanhos de cada linha/coluna)

```
.grid-container {
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-template-rows: 100px
  200px;
}
```



2 colunas que ocupam a mesma fração do espaço disponível (1 fr: uma fração do espaço);

2 linhas, uma com 100px de altura e a segunda com 200px;

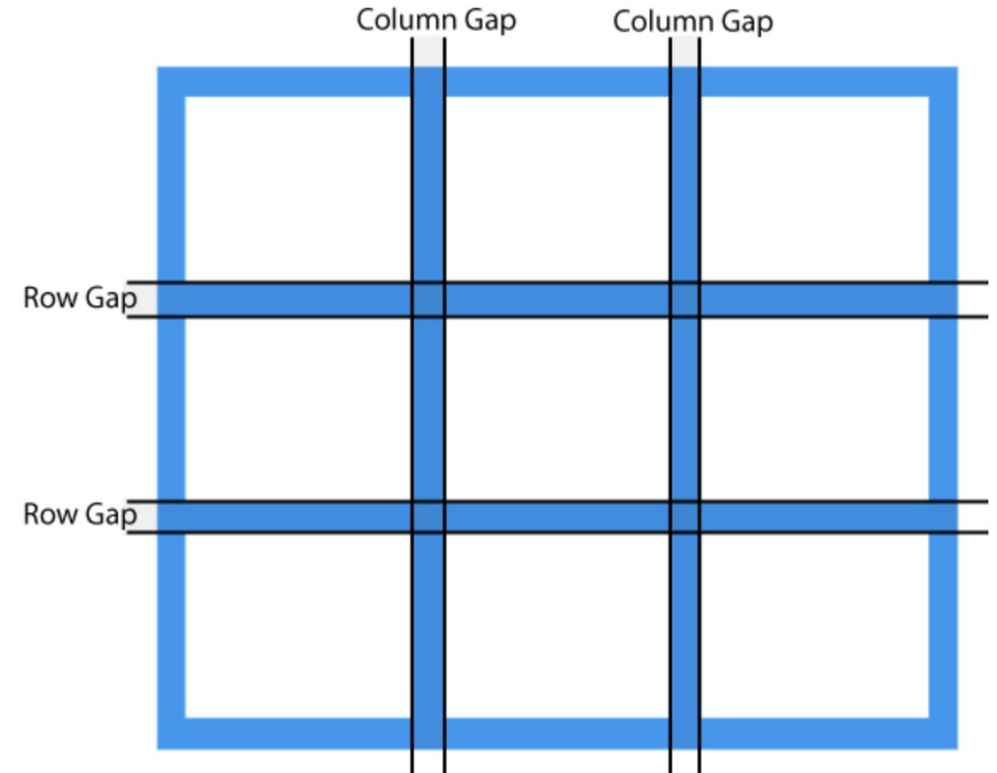
Display: grid

Espaçamento entre linhas e colunas

column-gap ; **row-gap** ; **gap**

- Os espaços entre cada coluna/linha são chamados *gaps*
- Podemos ajustar o tamanho do “gap” através destas 3 propriedades

```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  gap: 20px;  
}
```

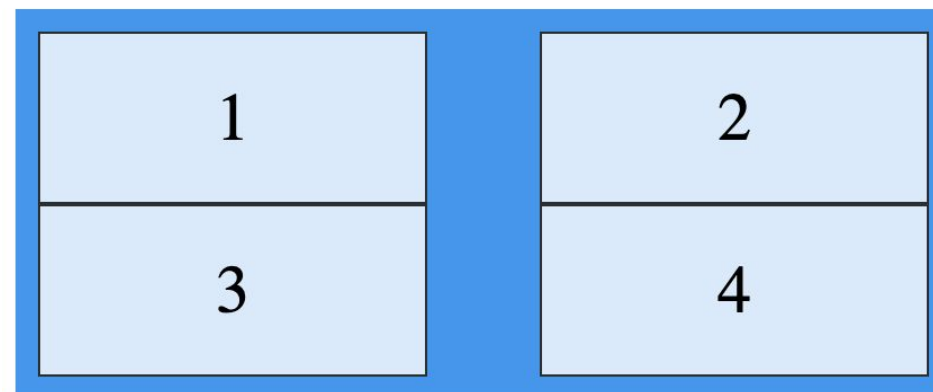


Display: grid

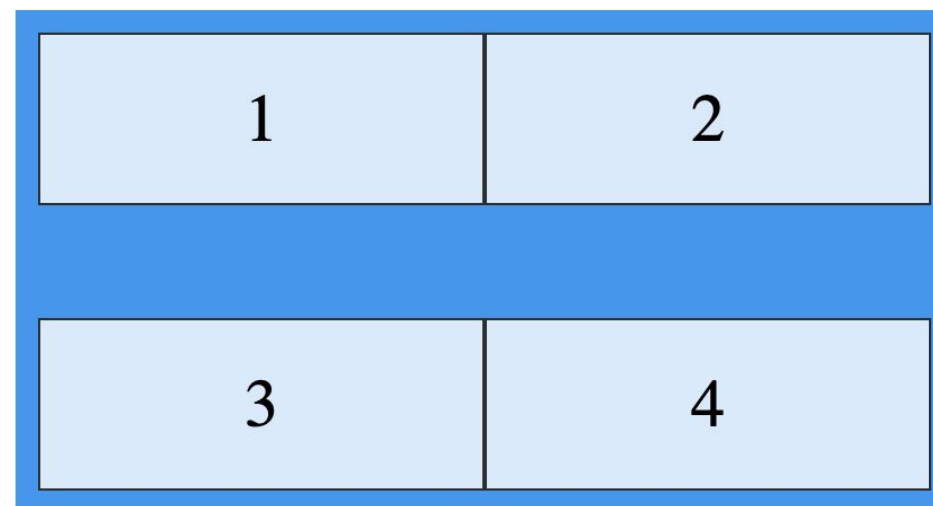
Espaçamento entre linhas e colunas

column-gap ; **row-gap** ; **gap**

```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  column-gap: 20px;  
}
```



```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  row-gap: 20px;  
}
```



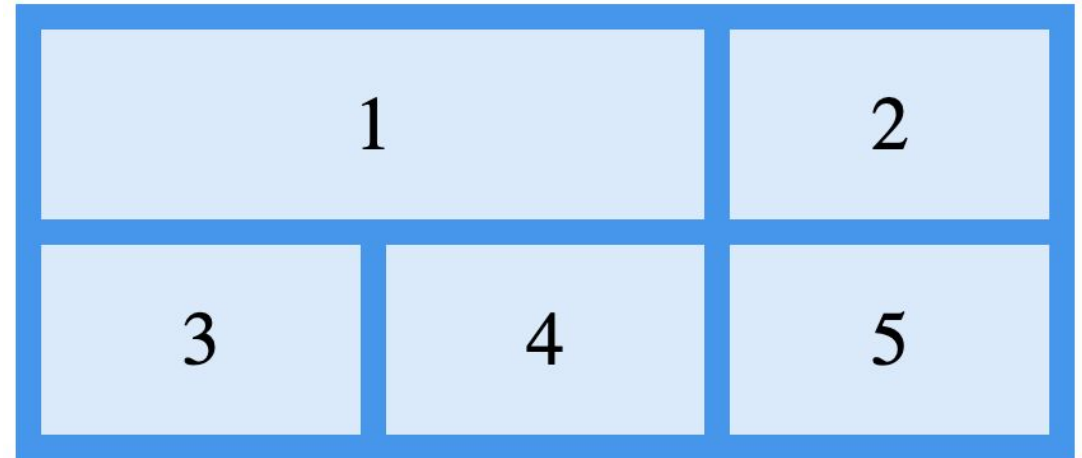
Display: grid

Posicionamento dos elementos dentro do *container*

grid-column ; **grid-row**

```
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto;
  gap: 10px;
}
```

```
.item1 {
  grid-column-start: 1;
  grid-column-end: 3;
}
```



grid-column: 1 / 3;

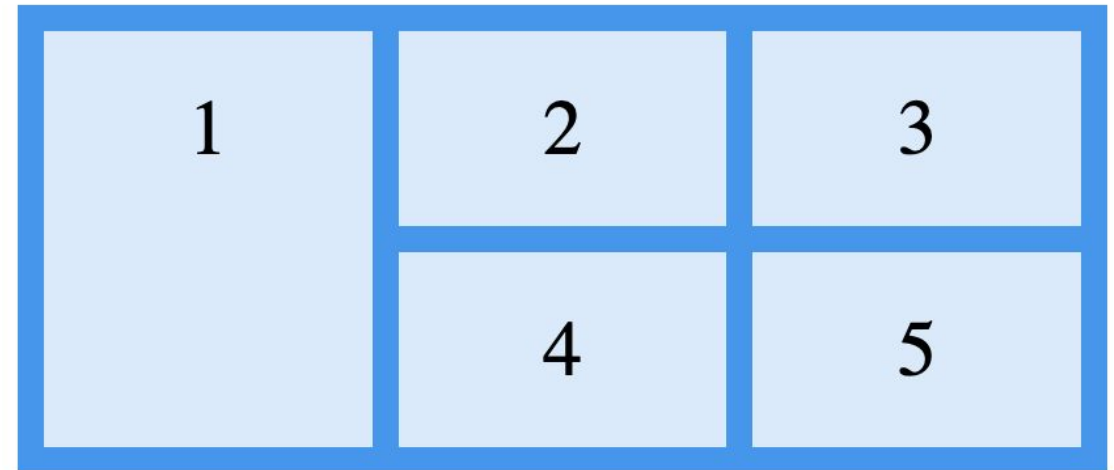
Display: grid

Posicionamento dos elementos dentro do *container*

grid-column ; **grid-row**

```
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto;
  gap: 10px;
}
```

```
.item1 {
  grid-row-start: 1;
  grid-row-end: 3;
}
```



grid-row: 1 / 3;

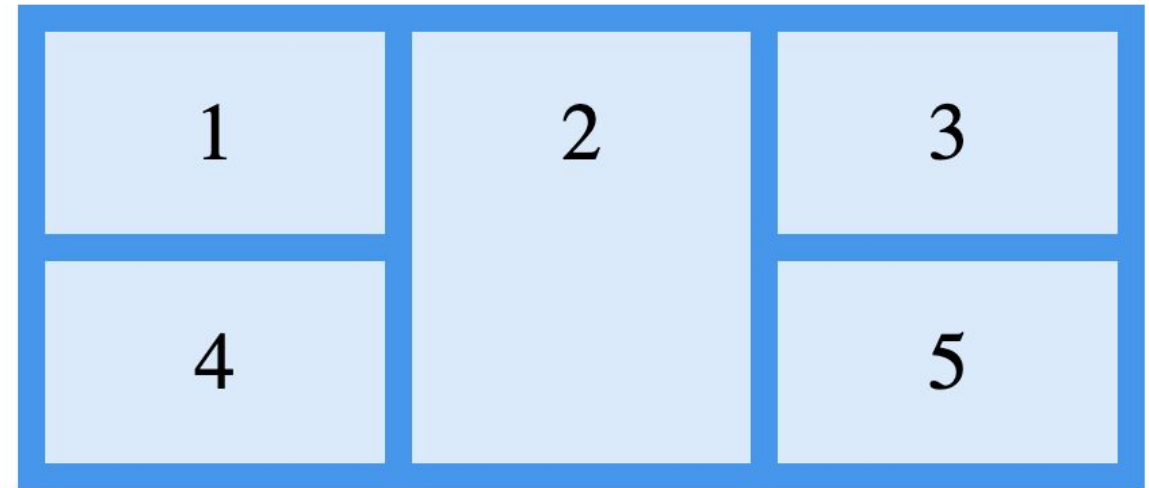
Display: grid

Posicionamento dos elementos dentro do *container*

grid-column ; **grid-row**

```
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto;
  gap: 10px;
}
```

```
.item2 {
  grid-row-start: 1;
  grid-row-end: 3;
  grid-column: 2 / 3;
}
```



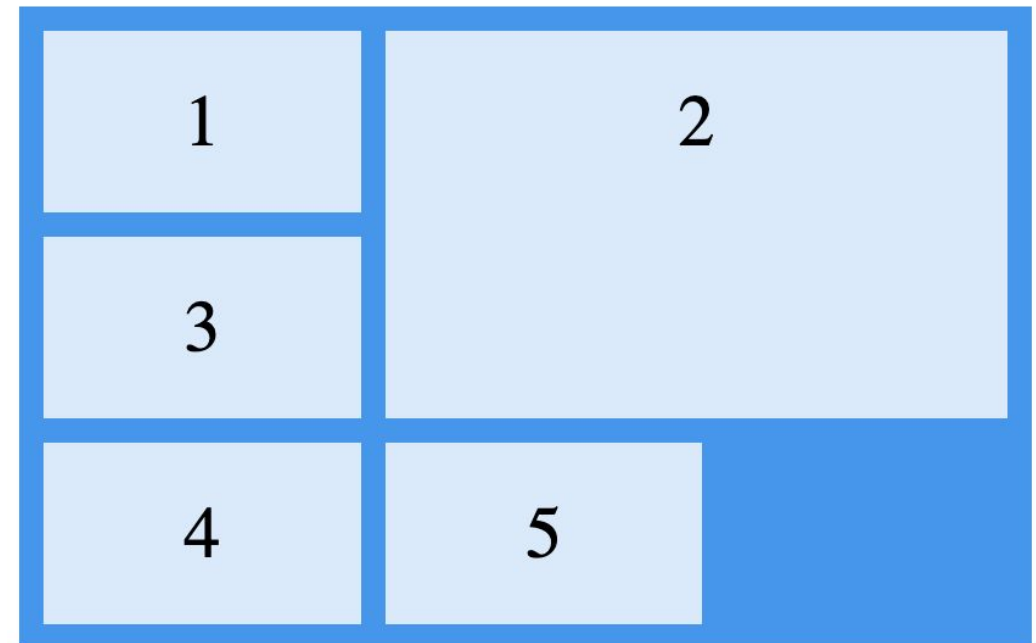
Display: grid

Posicionamento dos elementos dentro do *container*

grid-column ; **grid-row**

```
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto;
  gap: 10px;
}
```

```
.item2 {
  grid-row-start: 1;
  grid-row-end: 3;
  grid-column: 2 / 4;
}
```

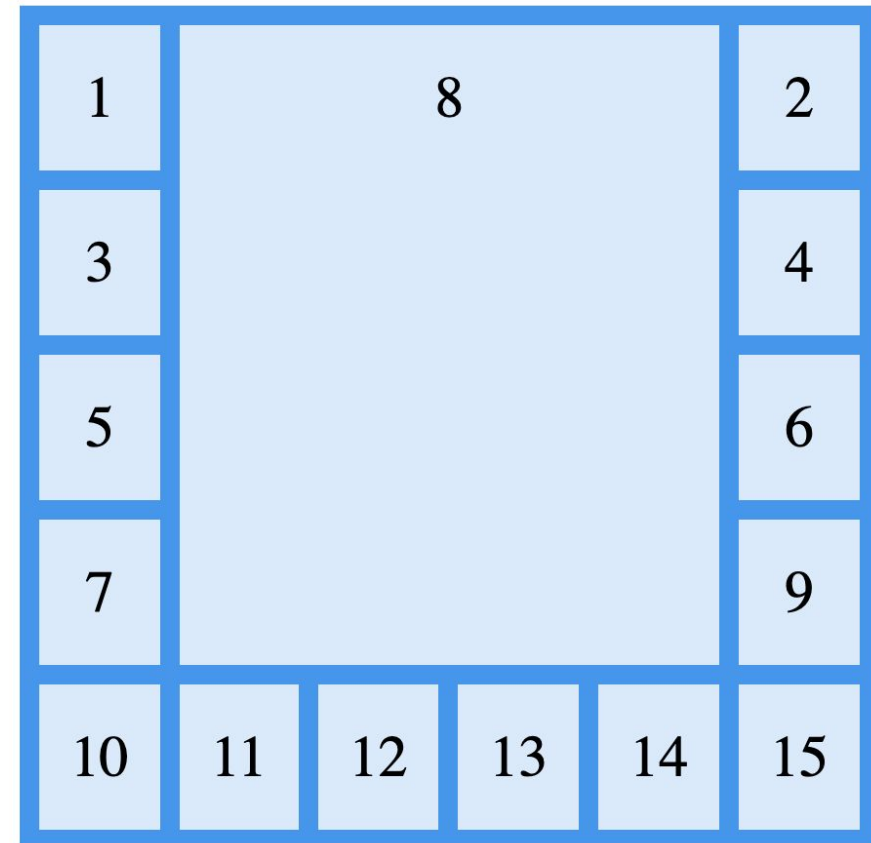


Display: grid

grid-area grid-row-start / grid-column-start / grid-row-end / grid-column-end

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(6, auto);
  gap: 10px;
}

.item8 {
  grid-area: 1 / 2 / 5 / 6;
}
```



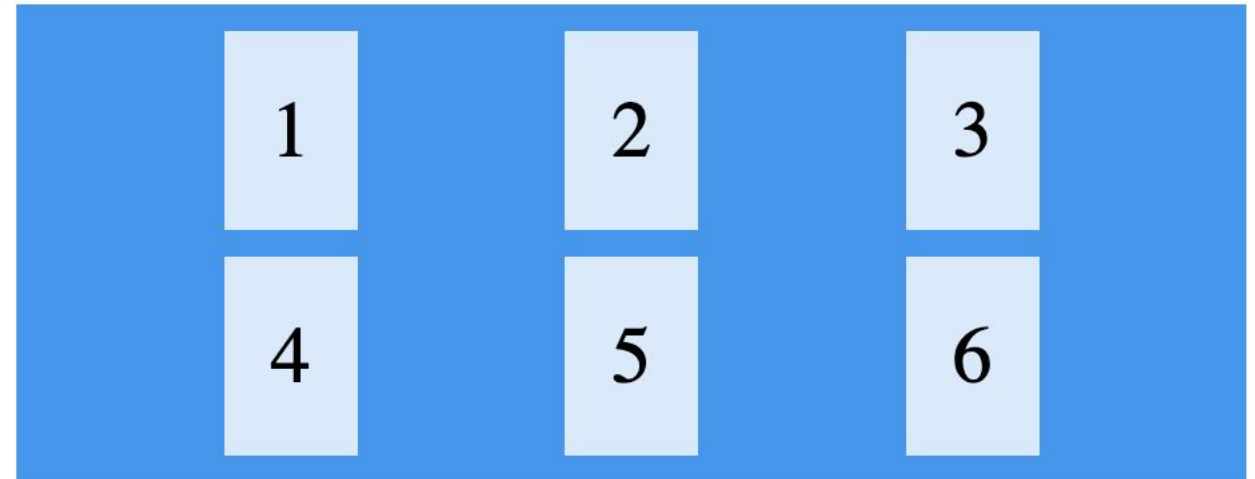
Display: grid

justify-content space-evenly / space-around / space-between / center / start / end

Alinhar **horizontalmente** os elementos dentro do container *grid*

```
.grid-container {
  display: grid;
  grid-template-columns: 50px 50px 50px;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
  justify-content: space-evenly;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  text-align: 30px;
}
```



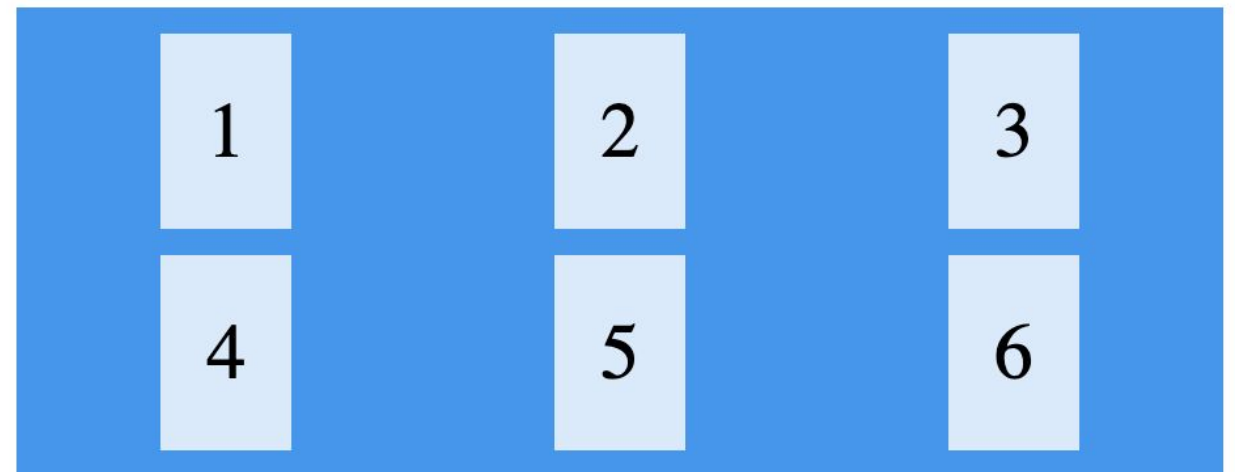
Display: grid

justify-content space-evenly / space-around / space-between / center / start / end

Alinhar **horizontalmente** os elementos dentro do container *grid*

```
.grid-container {
  display: grid;
  grid-template-columns: 50px 50px 50px;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
  justify-content: space-around;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  text-align: 30px;
}
```



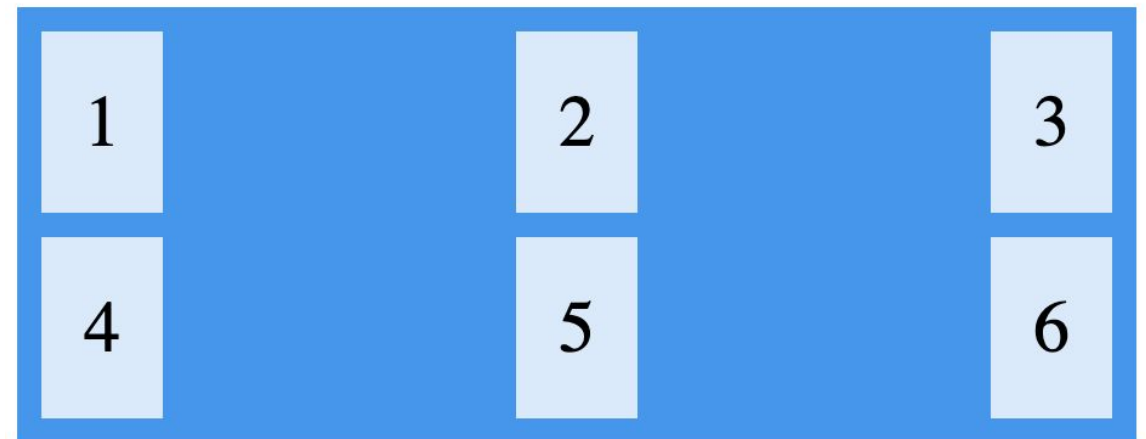
Display: grid

justify-content space-evenly / space-around / **space-between** / center / start / end

Alinhar **horizontalmente** os elementos dentro do container *grid*

```
.grid-container {
  display: grid;
  grid-template-columns: 50px 50px 50px;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
  justify-content: space-between;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  text-align: 30px;
}
```



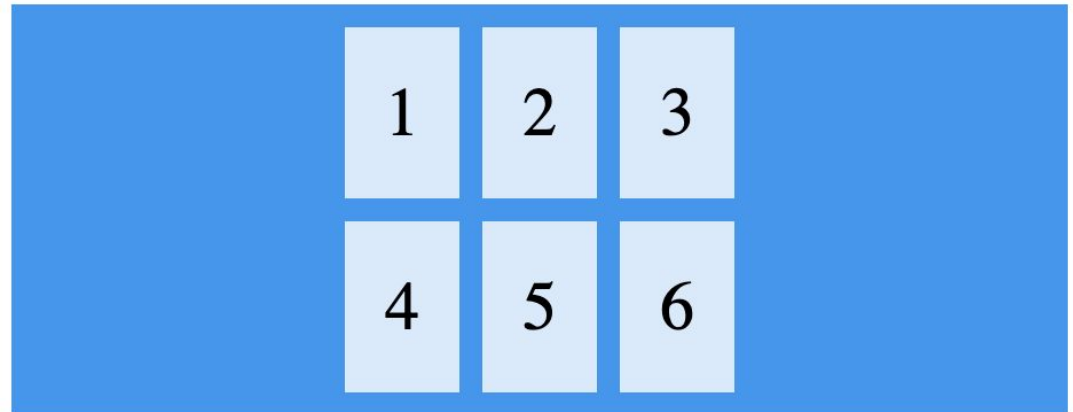
Display: grid

justify-content space-evenly / space-around / space-between / center / start / end

Alinhar **horizontalmente** os elementos dentro do container *grid*

```
.grid-container {
  display: grid;
  grid-template-columns: 50px 50px 50px;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
  justify-content: center;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  text-align: 30px;
}
```



Display: grid

justify-content space-evenly / space-around / space-between / center / **start** / end

Alinhar **horizontalmente** os elementos dentro do container *grid*

```
.grid-container {
  display: grid;
  grid-template-columns: 50px 50px 50px;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
  justify-content: start;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  text-align: 30px;
}
```



Display: grid

justify-content space-evenly / space-around / space-between / center / start / end

Alinhar **horizontalmente** os elementos dentro do container *grid*

```
.grid-container {
  display: grid;
  grid-template-columns: 50px 50px 50px;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
  justify-content: end;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  text-align: 30px;
}
```

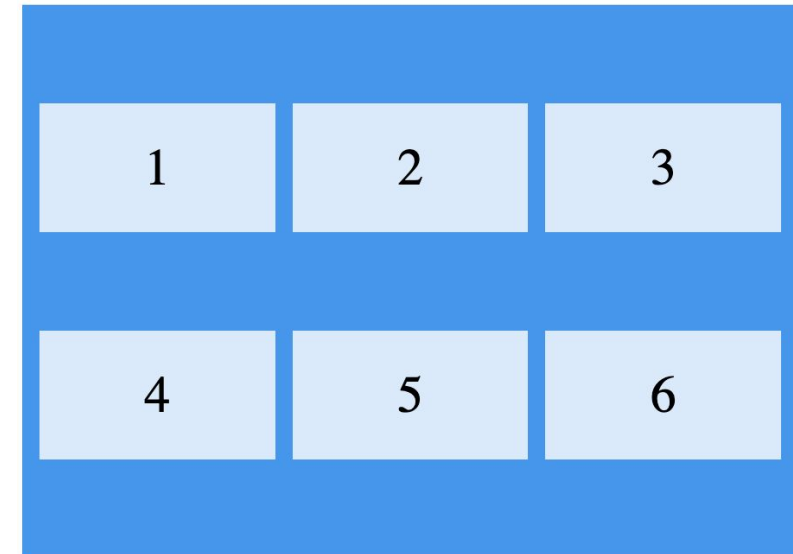


Display: grid

align-content space-evenly / space-around / space-between / center / start / end

Alinhar **verticalmente** os elementos dentro do container *grid*

```
.grid-container {
  display: grid;
  height: 400px;
  grid-template-columns: auto auto auto;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
  align-content: space-evenly;
}
```

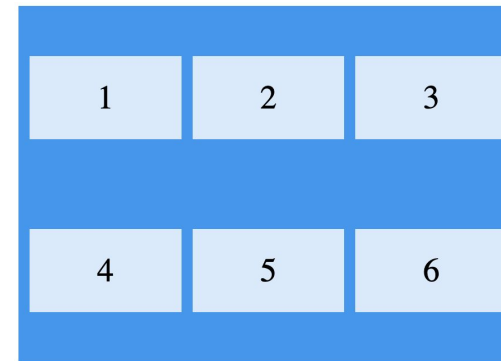


Display: grid

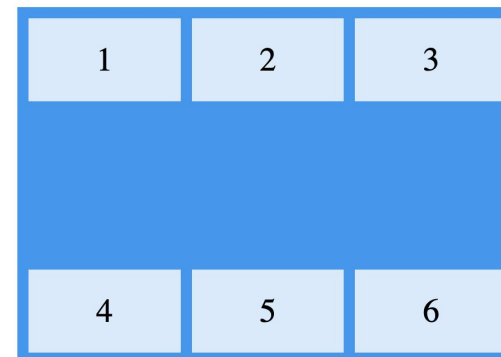
align-content space-evenly / space-around / space-between / center / start / end

Alinhar **verticalmente** os elementos dentro do container *grid*

```
align-content: space-around;
```



```
align-content: space-between;
```



Display: grid

align-content space-evenly / space-around / space-between / center / start / end

Alinhar **verticalmente** os elementos dentro do container *grid*

`align-content: center;`

1	2	3
4	5	6

`align-content: start;`

1	2	3
4	5	6

`align-content: end;`

1	2	3
4	5	6

Exercício

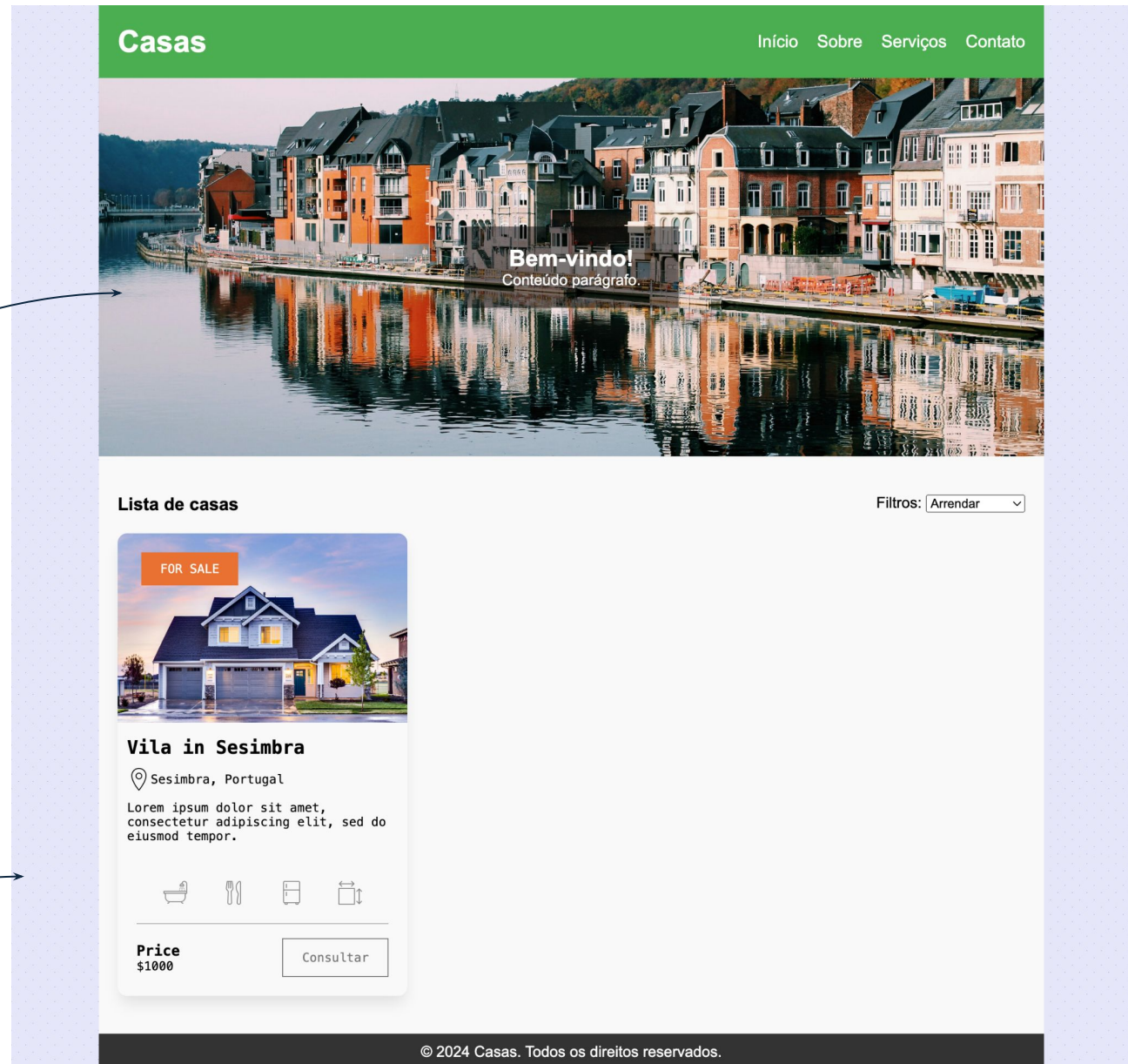
Home Page

- Container com header, main e footer, usando propriedade display: grid;
- Header color: #4CAF50
- Footer color: #333333

Section com background-image e elementos posicionados ao centro com Flexbox

body com background-image

max-width: 1000px

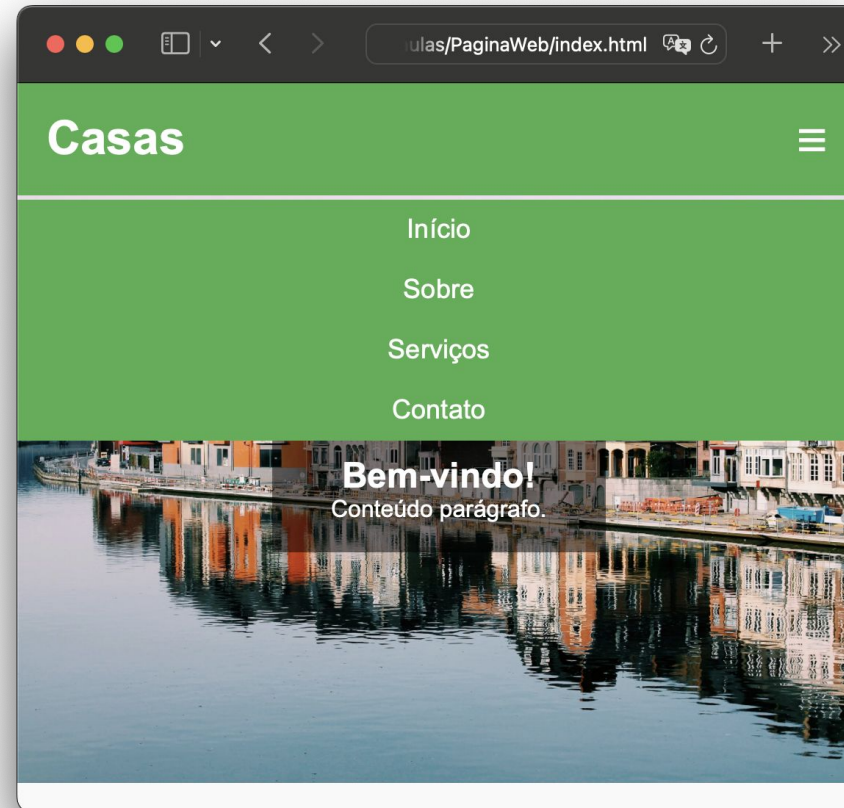


Exercício

<nav>, com class: “nav”
, com class: “nav-list”

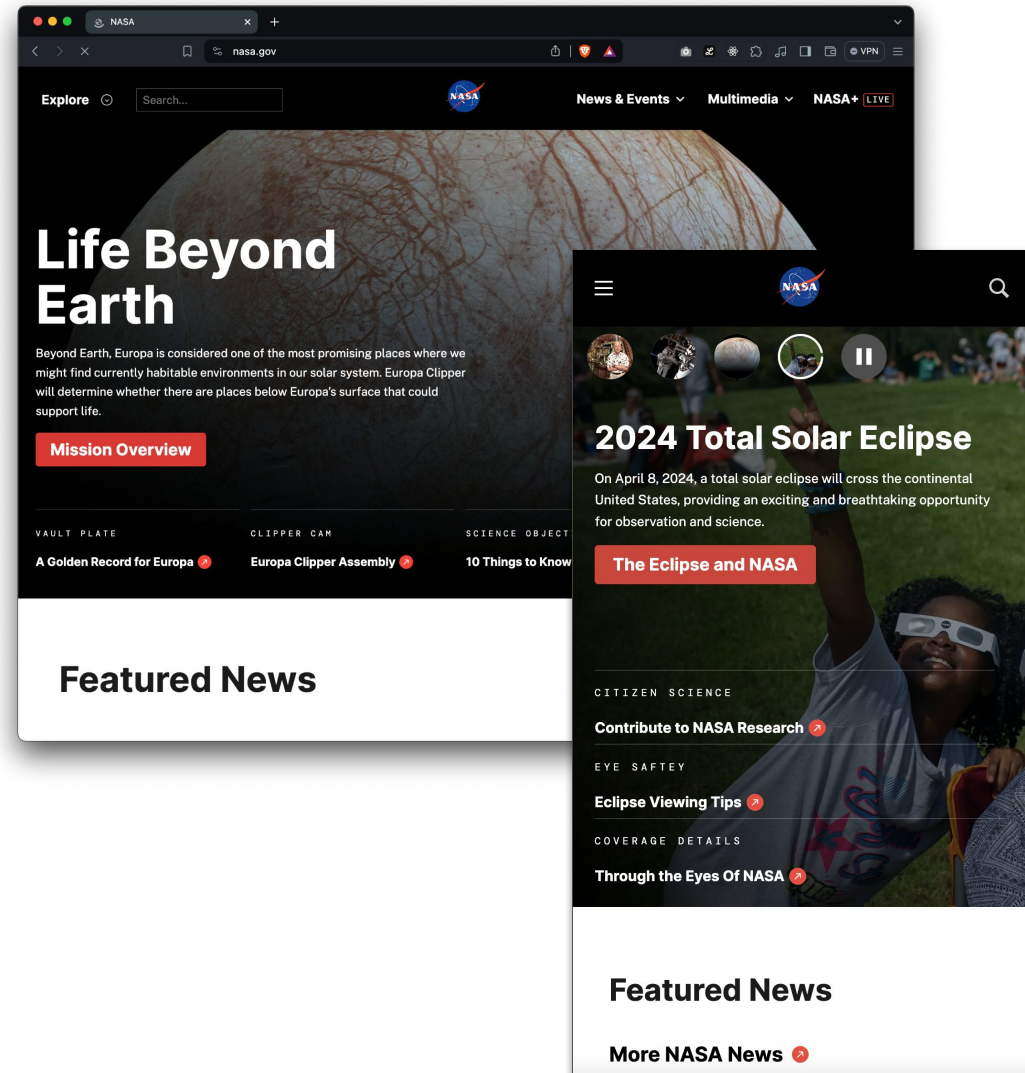
Incluir o seguinte <div> dentro do
<header>

```
<div class="menu-toggle" id="menu-toggle">  
  ≡  
</div>
```



Media Queries

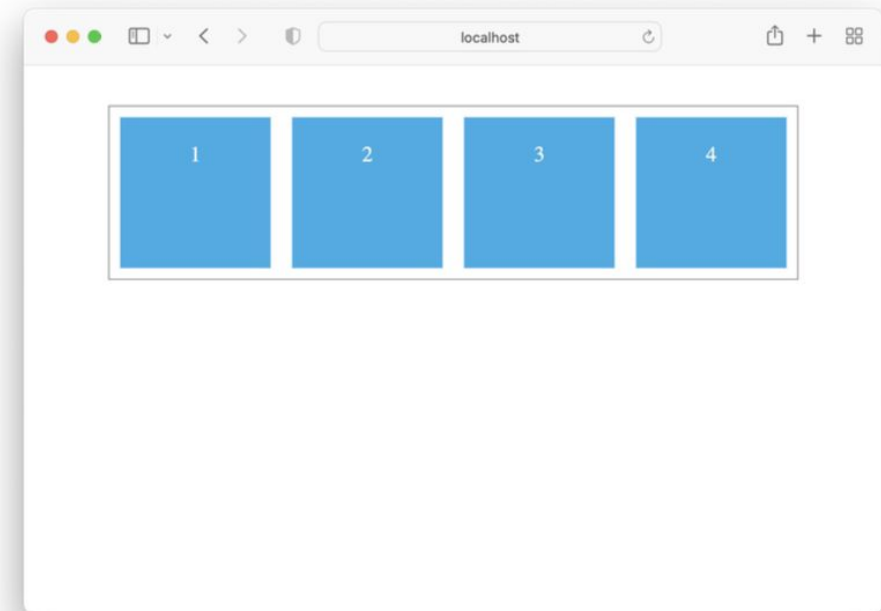
- Como podemos programar a nossa página para aparecer bem em todos os ecrãs?
- Utilizar larguras máximas não é suficiente e é necessário definir condições concretas para como o layout é apresentado em ecrãs de dimensões diferentes.



Media Queries

- É habitual que uma página apresente uma disposição horizontal de elementos em ecrãs mais largos.
- Como podemos alinhar aqueles elementos verticalmente quando o ecrã fica menos largo?

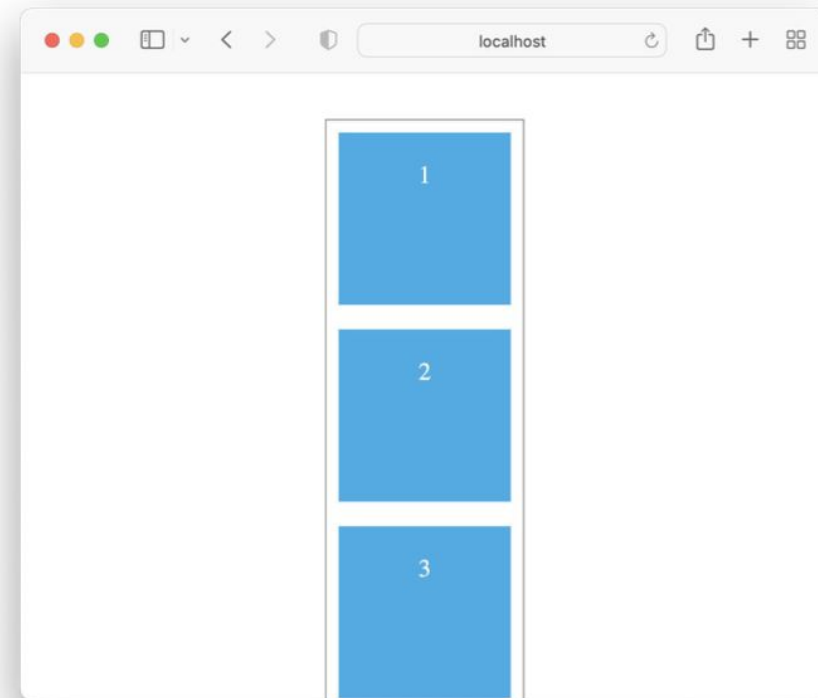
```
.container {  
  position: relative;  
  display: flex;  
  border: #8B8B8B solid 1px;  
  margin: 30px;  
  gap: 10px;  
  padding: 10px;  
}
```



Media Queries

```
.container {  
  position: relative;  
  display: flex;  
  border: #8B8B8B solid 1px;  
  margin: 30px;  
  gap: 10px;  
  padding: 10px;  
}  
  
@media (max-width: 768px) {  
  .container {  
    flex-direction: column;  
  }  
}
```

width <= 768px



Media Queries

- Existe um conjunto universal de valores que definem as quebras nas páginas

Dispositivos muito pequenos (600px ou menos)	<code>@media screen and (max-width: 600px) { . . . }</code>
Dispositivos pequenos (600px ou mais)	<code>@media screen and (min-width: 600px) { . . . }</code>
Dispositivos médios (768px ou mais)	<code>@media screen and (min-width: 768px) { . . . }</code>
Dispositivos grandes (992px ou mais)	<code>@media screen and (min-width: 992px) { . . . }</code>
Dispositivos muito grandes (1200px ou mais)	<code>@media screen and (min-width: 1200px) { . . . }</code>

Media Queries

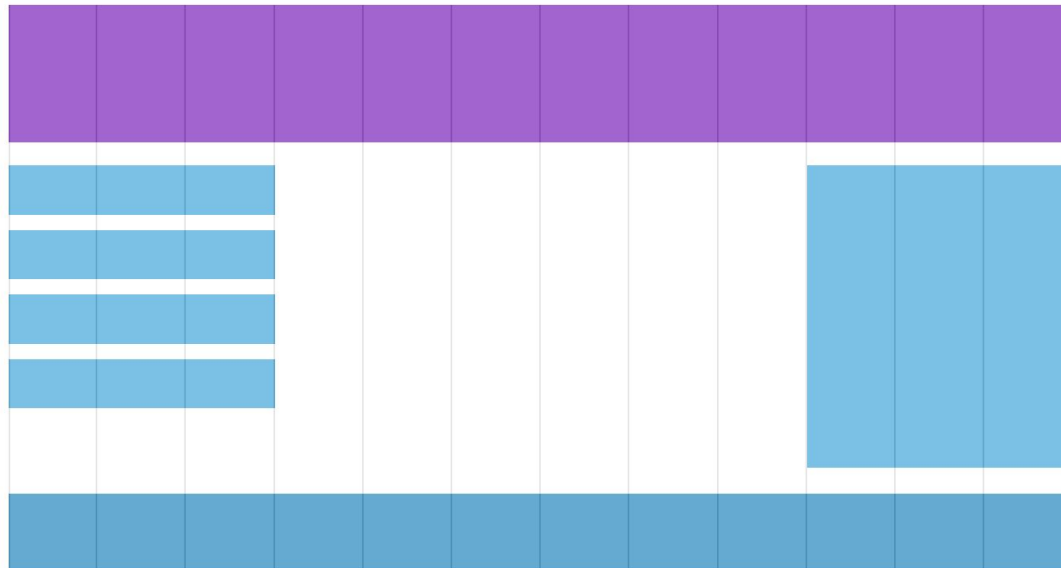
- Para a página ser corretamente apresentada nos dispositivos móveis é necessário adicionar a seguinte tag <meta> dentro do head:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- Caso esta linha não seja utilizada, o dispositivo vai considerar uma largura de janela maior e poderá não aplicar as regras de responsabilidade.

Media Queries

- Para simplificar o sistema, podemos dividir a página em colunas - **Grid-view**.



```
.col-1 {width: 8.33%;}  
.col-2 {width: 16.66%;}  
.col-3 {width: 25%;}  
.col-4 {width: 33.33%;}  
.col-5 {width: 41.66%;}  
.col-6 {width: 50%;}  
.col-7 {width: 58.33%;}  
.col-8 {width: 66.66%;}  
.col-9 {width: 75%;}  
.col-10 {width: 83.33%;}  
.col-11 {width: 91.66%;}  
.col-12 {width: 100%;}
```


- Cross-browser: garantir a compatibilidade com todos os dispositivos e com diferentes browsers. Muitas vezes temos de implementar código diferente para diferentes browsers.
- Responsive Web Design: O layout do website deve adaptar-se automaticamente aos diferentes tamanhos de ecrã dos dispositivos; Muitas vezes temos de mudar o layout da página ou remover texto/images;
- Performance: Páginas Web devem ser rápidas carregar. Escolher tamanho ideal de imagens; minimizar o código; etc.
- Accessibility: Website deve ser acessível a todos - adaptar conteúdo a pessoas com deficiências visuais, deficiências auditivas, incapacidades cognitivas ou incapacidades físicas.
- Internationalization
- Privacy & Security

- Usar elementos HTML de forma a fornecer significado estrutural e contextual para o conteúdo da página.
- Em vez de simplesmente usar elementos HTML com base na sua aparência visual, selecionar cuidadosamente os elementos que melhor descrevem a estrutura e conteúdo. - **HTML Semantic**
- Elementos que descrevem melhor o conteúdo: `<header>`, `<nav>`, `<article>`, `<section>`, `<footer>` - em vez de usar só `<div>` para todo o conteúdo
- SEO: mecanismos de busca utilizam a semântica HTML apropriados para entender melhor o contexto do conteúdo e melhor no posicionamento dos resultados de pesquisa
- Cada elemento tem um propósito semântico claro que ajuda na manutenção do website

Práticas Web - exemplo

<header>

<section>

<footer>

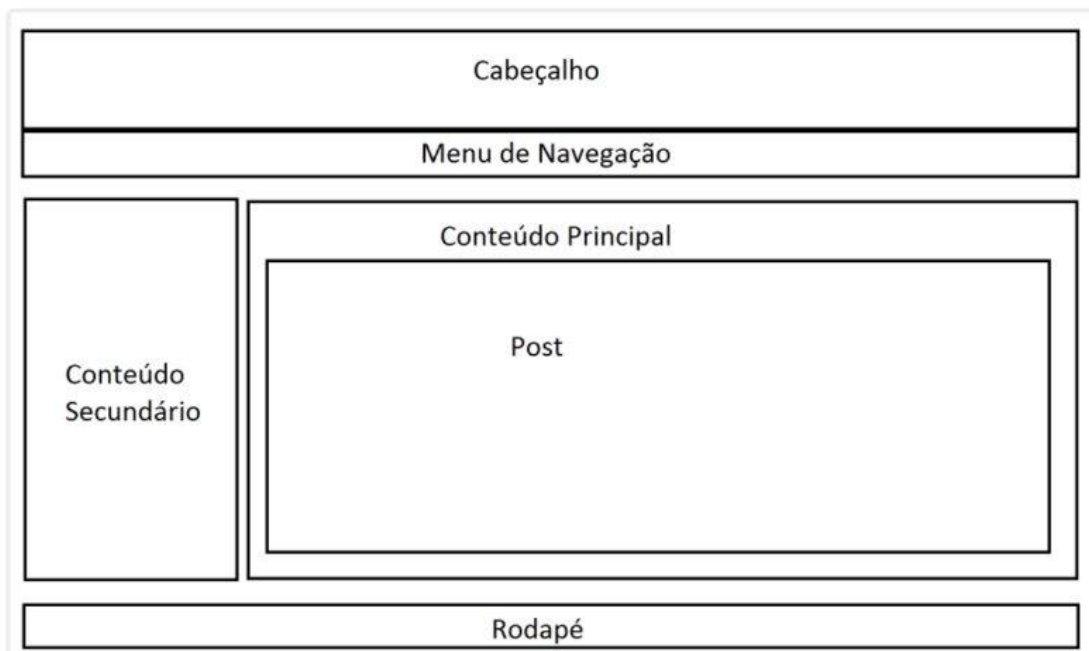


```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exemplo de HTML Semântico</title>
</head>
<body>
  <header>
    <h1>Meu Site</h1>
    <nav>
      <ul>
        <li><a href="#">Início</a></li>
        <li><a href="#">Sobre</a></li>
        <li><a href="#">Contato</a></li>
      </ul>
    </nav>
  </header>

  <section>
    <article>
      <h2>Título do Artigo</h2>
      <p>Conteúdo do artigo...</p>
    </article>
  </section>

  <footer>
    <p>&copy; 2024 Meu Site. Todos os direitos reservados.</p>
  </footer>
</body>
</html>
```

Práticas Web

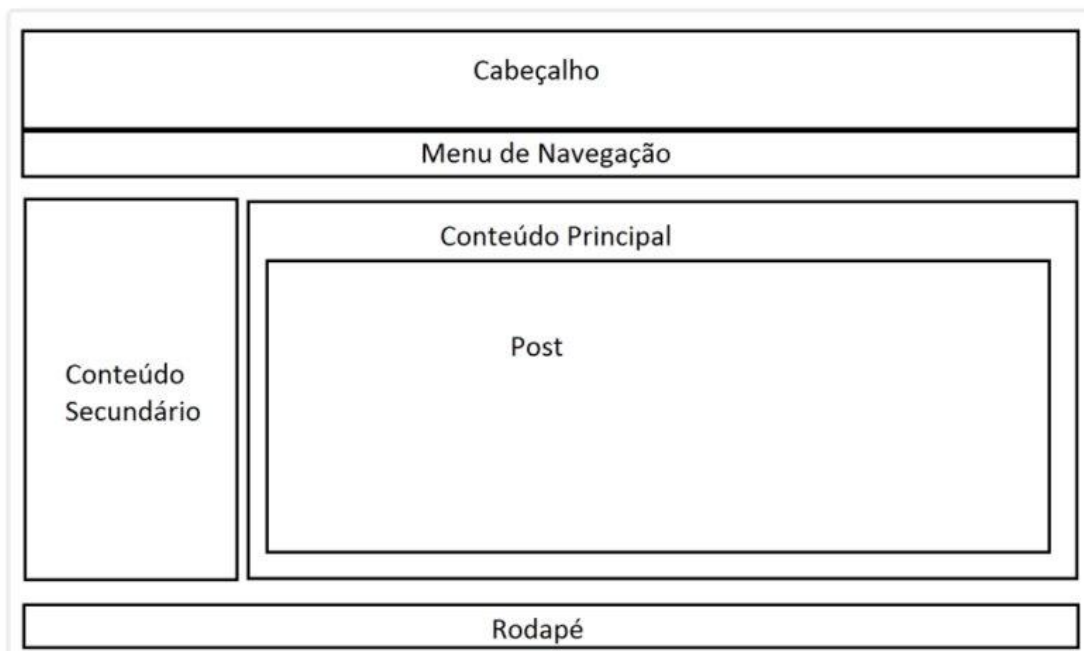


```
<div class="cabecalho" >
|   <div class="menu-navegacao" ></div>
</div>

<div class="conteudo-secundario" ></div>

<div class="conteudo-principal" >
|   <div class="post" ></div>
</div>

<div class="rodape" ></div>
```



```
<header class="cabecalho" >
|   <nav class="menu-navegacao" ></nav>
</header>

<aside class="conteudo-secundario" ></aside>

<main class="conteudo-principal" >
|   <article class="post" ></article>
</main>

<footer class="rodape" ></footer>
```

Práticas Web - Semântica

Div e Span: <div> não exibe nada, a menos que a usando CSS. As demais tags também são assim. Servem apenas para fazermos divisões.

- <div> é um elemento de bloco
- é um elemento de linha

Article

Define um artigo. Muito utilizado em Posts de blogs e sites de notícias.

Aside

Define um conteúdo ao lado do conteúdo da página. Muito utilizado em colunas ao lado da parte principal dos sites, onde normalmente possui a navegação, propagandas ou algo relacionado com o conteúdo principal.

Code

Define um pedaço de código de computador

Práticas Web - Semântica

Details e Summary

A tag <details> serve para acrescentar detalhes adicionais sobre algo, mas essa informação fica escondida, sendo exibida apenas quando o usuário clicar para ler.

A tag <summary> serve para exibir um título para o <details>, que ficará sempre visível.

Dialog

serve para criar modais (popups).

Footer

Define um rodapé para o documento ou seção.

Header

Define um cabeçalho para o documento ou seção.

Main

Define a parte principal de um documento. Por ser algo indicado como principal, o conteúdo dessa tag deve ser único e ela não deve ser filha de tags como <article>, <footer>, <header> ou <nav>.

Nav

Define um grupo de links de navegação. Essa tag é muito utilizada nos menus utilizados para se navegar entre as páginas de um site.

Section

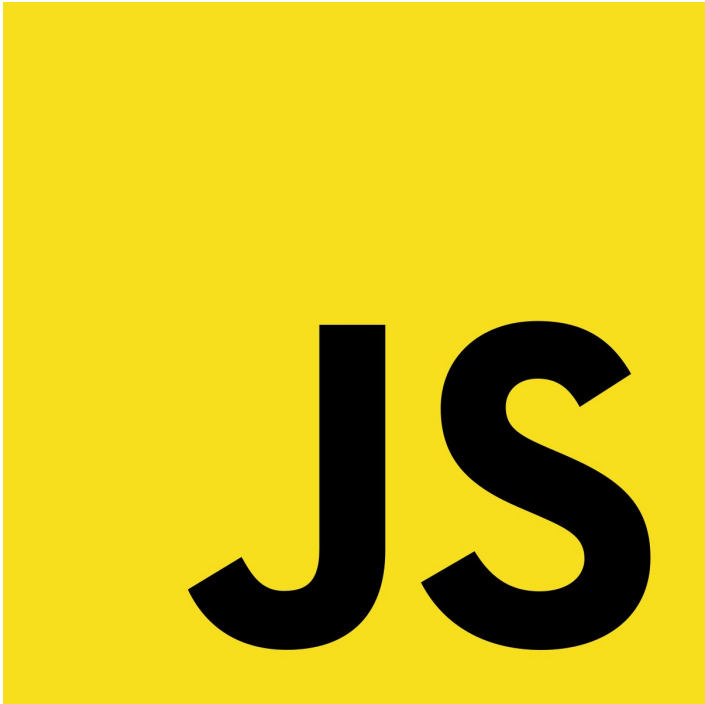
Funciona como a Div, sendo mais específica. Define seções no documento, como capítulos, cabeçalhos, rodapés, etc.

Javascript

Linguagem de programação, orientada a objetos, amplamente utilizada para desenvolver aplicações web.

Javascript permite manipular o conteúdo da página, responder a eventos, fazer chamadas assíncronas ao servidor, e interação.

Linguagem interpretada pelos navegadores Web - executado no lado do cliente

The image shows the JavaScript logo, which consists of the letters 'J' and 'S' in a bold, black, sans-serif font. The letters are positioned on a solid yellow square background. The 'J' is on the left and the 'S' is on the right, with a small gap between them.

Running and Loading JS

1. Adicionar código JS diretamente nos atributos de um elemento HTML:

```
<input type="button" value="click" onClick='alert("Button Clicked");' />
```

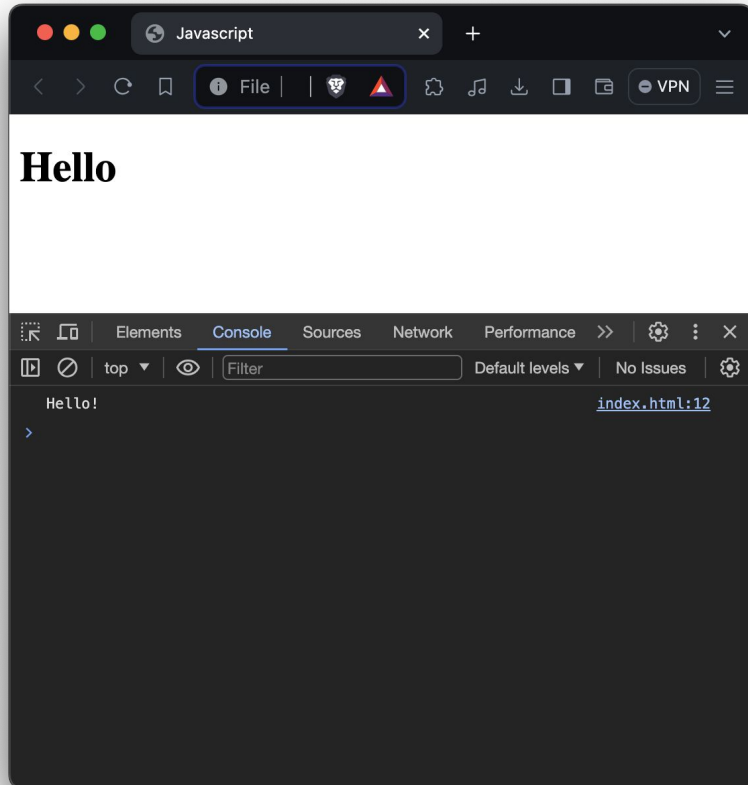
2. No ficheiro HTML, dentro de um elemento <script>:

```
<script type="text/javascript">  
    alert("Script Tag!!");  
</script>
```

3. Ficheiro Javascript:

```
<script src="./file.js"></script>
```

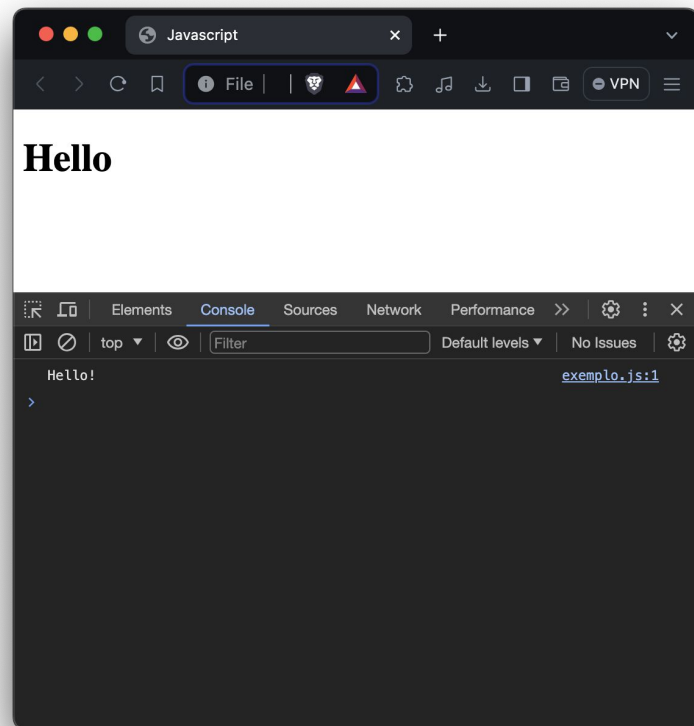
Running and Loading JS



```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width
  <title>Javascript</title>
</head>
<body>
  <h1>Hello</h1>

  <script>
    console.log("Hello!");
  </script>
</body>
</html>
```

Running and Loading JS

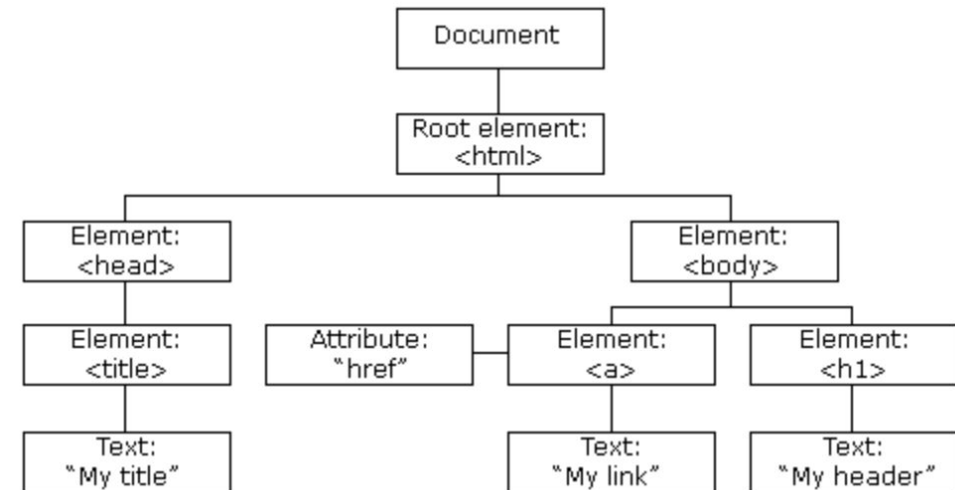


```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-wi
  <title>Javascript</title>
</head>
<body>
  <h1>Hello</h1>

  <script src="exemplo.js"></script>
</body>
</html>
```

DOM - Document Object Model

- Representação em forma de árvore de todos os elementos de uma página Web.
- Objeto e interface para interagir e manipular os elementos HTML.
- Objeto que permite obter, alterar, adicionar ou eliminar elementos HTML;
- *document*.



DOM - Document Object Model

- Operações DOM:
 - Seleccionar elementos HTML;

```
let elementoHTML = document.getElementById('elementoID');
```

- Alterar ou manipular conteúdo;

```
elementoHTML.textContent = 'Novo texto';  
elementoHTML.classList.add('nova_classe');
```

- Manipular eventos;

```
elementoHTML.addEventListener('click', function() {  
    console.log('Clicou no elemento');  
});
```

DOM - Document Object Model

- Operações DOM:
 - Alterar estilos CSS;

```
elementoHTML.style.color = 'red';
```

- Criar novos elementos HTML

```
let elementoHTML = document.createElement('div');  
document.body.appendChild(elementoHTML);
```

Manipular elementos HTML

```
<div class="menu">
  <h1 id='elementoID'>Titulo</h1>
  <p>Fotografias</p>
  <p>Música</p>
</div>
```

```
// obter elemento pelo seu ID
let elementoHTML = document.getElementById('elementoID');

// obter lista de elementos pela class:
let elementoHTML = document.getElementsByClassName('elementos');

// obter lista de elementos pela tag:
let elementoHTML = document.getElementsByTagName('p');

// seleccionar elemento por classe ou ID:
let elementoHTML = document.querySelectorAll('.class_elementos');

let elementoHTML = document.querySelector('#id_elemento');
```

Alterar estilos de elementos HTML

- Depois de termos acesso ao elemento específico, podemos manipular os seus estilos, classes, etc.
- A propriedade *style* permite-nos aceder ou alterar os estilos de um elemento.

```
<h1 id="titulo">Titulo</h1>
```

```
// obter elemento
let elemento_titulo = document.getElementById('titulo');

console.log(elemento_titulo.style.fontSize);

// alterar CSS atributos/valores
elemento_titulo.style.fontSize = "20px";
```


Manipular Eventos

- Existem diversos tipos de eventos que podem ser capturados e tratados em um página Web.
- Estes eventos podem invocar uma função associada.

Eventos de Mouse:

- **click**: users clica em um elemento;
- **dblclick**: user realiza um duplo clique em um elemento;
- **mouseover**: cursor do *mouse* entra em um elemento;
- **mouseout**: cursor do *mouse* sai de um elemento;

Eventos de Teclado:

- **keydown**: tecla do teclado pressionada;
- **keypress**: tecla é pressionada e largada;

Eventos de Formulário:

- **submit**: formulário submetido;
- **change**: valor de input do formulário alterado;

Eventos de Janela:

- **load**: ocorre após a janela ou elemento é carregado;
- **resize**: ocorre quando a janela do navegador é redimensionada;
- **scroll**: ocorre quando o conteúdo faz *scroll*;

Eventos de Tempo:

- **setTimeout**: função executada uma vez após um intervalo de tempo específico;
- **setInterval**: função executada repetidamente em intervalos de tempo específicos;

Manipular Eventos - exemplo

- Método `addEventListener()`
 - permite “anexar” um evento a um elemento HTML
 - podemos adicionar múltiplos eventos a um elemento

```
<h1 id="titulo">Titulo</h1>
```

```
let elemento_titulo = document.getElementById('titulo');

elemento_titulo.addEventListener('click', function() {
    console.log('Botão pressionado.');
```

Manipular elementos - cont.

Podemos seleccionar todos os elementos de uma classe através do método `.querySelectorAll`;
 Para iterar sobre os elementos usamos o método *forEach*

```
<div class="class_div">Elemento 1</div>
<div class="class_div">Elemento 2</div>
<div class="class_div">Elemento 3</div>
```

```
let elementos = document.querySelectorAll('.class_div');

// remover class dos elementos
elementos.forEach(function(elemento) {
  elemento.classList.remove('class_div');
})
```

Manipular elementos - cont.

```
<div class="class_div">Elemento 1</div>  
<div class="class_div">Elemento 2</div>  
<div class="class_div">Elemento 3</div>
```

```
let elementos = document.querySelectorAll('.class_div');  
  
// adicionar class dos elementos  
elementos.forEach(function(elemento) {  
    elemento.classList.add('class_div');  
})
```

Criar elementos

- Podemos criar elementos HTML de forma dinâmica;
- Prática comum em Javascript quando existem dados dinâmicos ou interações do *user*.

```
<div id="container"></div>
```

```
let container = document.querySelector('#container');

// criar um novo elemento <p>
let paragrafo = document.createElement('p');

// Definir o conteúdo
paragrafo.textContent = 'Parágrafo criado dinamicamente!';

// Adicionar class ao elemento criado
paragrafo.classList.add('nova_class');

// Adicionar/anexar o elemento <p> ao <div>
container.appendChild(paragrafo);
```

Criar elementos

- Também podemos criar a nossa estrutura dentro de uma string com interpolação de variáveis ou expressões:

```
let nome = 'João';
let idade = 20;

let html = `
  <div class="user">
    <p>Nome: ${nome}</p>
    <p>Idade: ${idade}</p>
  </div>
`;

let container = document.getElementById('container');

// Inserir HTML ao elemento selecionado
container.innerHTML = html;
```

Exercício

1. Adicionar um evento ao elemento `h1`, para mudar a cor para vermelho ao clicar. Nota: usar método `toggle`
2. Adicionar um evento ao elemento `div`, que muda a cor de background quando o cursor para por cima.
Nota: usar evento: `mouseover` e `mouseout`;
3. Somar dois números. `<input>`
4. Mudar cor de elementos com a mesma class.

Mudar cor do Titulo ao 'click'

Mudo a cor quando o cursor passa por cima.

Somar dois números inteiros

Mudar cor individual de cada elemento com a mesma Classe

Elemento 1

Elemento 2

Elemento 3

Aceder parâmetros no URL

- Uma das formas de passar informação entre páginas é definir parâmetros no URL.

`https://iscte-iul.pt/about.html?nome=Antonio&apelido=Santos`

- A estrutura desses parâmetros é a seguinte:

```
?campo1=valor1&campo2=valor2&campo3=valor3
```


Aceder parâmetros no URL

- Para aceder a cada um dos valores, podemos usar o *URLSearchParams*

`https://iscte-iul.pt/about.html?nome=Antonio&apelido=Santos`

```
let parametros= new URLSearchParams(window.location.search)

console.log(parametros.get("nome"))
>> "Antonio"

console.log(parametros.get("apelido"))
>> "Santos"
```

- Espaço em memória que guarda um valor de um determinado tipo
- Características de uma variável:
 - Tipo
 - Nome
 - Valor
- Em Javascript, ao definir uma variável estamos a indicar o seu nome e o seu valor inicial. O tipo é inferido pelo valor.

- Ao contrário de outras linguagens de programação, em Javascript o tipo da variável é flexível, o que nos permite desenvolver o código sem ter de pensar e atribuir um tipo ao definir a variável;
- Usamos o operador de atribuição (=) para definir ou alterar o valor das variáveis;

```
let a = 8;  
let b = 'texto';  
let c = 2;  
  
a = c + 2;  
  
// a = 4
```

Tipos de dados

- Definem as operações que podem ser feitas nos dados, o significado dos dados e a maneira como os valores desse tipo são armazenados.
- Existem diferentes tipos de dados:
 - Undefined: variável não foi definida
 - Null: variável definida mas não tem nenhum valor atribuído;
 - Numbers;
 - Strings;
 - Booleans;
 - Arrays;
 - Object

```
let inteiro = 8;
let texto = "ola";
let lista = [1, 2, 3];
let objeto = {
  nome: "João",
  idade: 30
};
```

Instruções e Blocos de Instrução

- Instrução: ação que pode mudar o estado de uma variável. Em Javascript, uma instrução termina sempre com um ponto-e-vírgula (;)
- Um bloco de instruções é um conjunto de instruções entre chavetas que será executado sequencialmente

```
let inteiro = 8;
```

```
{
  a = b + 2;
  b = b - 1;
}
```

Operadores Matemáticos

- Cálculo entre dois valores

+	Soma
-	Subtração
*	Multiplicação
/	Divisão

+=	Adiciona valor a ele próprio
++	Adiciona 1 valor
--	Subtrai 1 valor

Operadores Comparação

- Comparar se duas ou mais variáveis se comparam

===	é igual?
<	é menor?
<=	é menor ou igual?

!=	é diferente?
>	é maior?
>=	é maior ou igual?

Operadores Lógicos

&&	conjunção (e / and)
	disjunção (ou / or)
!	negação

Operadores - Exemplos

```
2 === 2 -> true
```

```
2 === 3 -> false
```

```
2 !== 3 -> true
```

```
2 < 3 -> true
```

```
2 >= 3 -> false
```

```
true && true -> true
```

```
true && false -> false
```

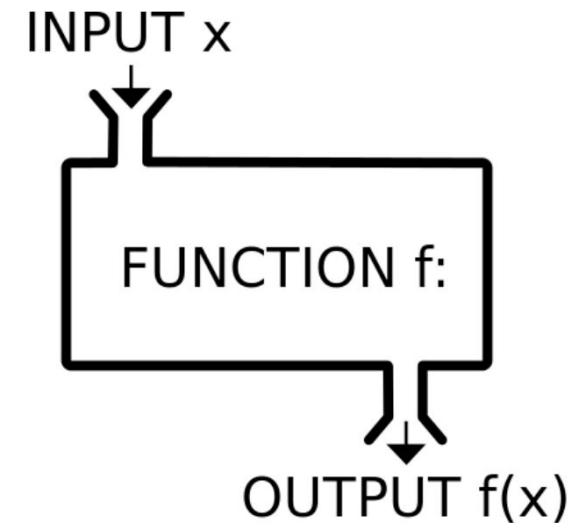
```
true || true -> true
```

```
false || true -> true
```

```
false || false -> false
```

Funções

- Bloco de instrução que performa uma tarefa. Essa tarefa só é executada se a função for chamada.
- Permite que blocos de código sejam reutilizados
- Pode ter argumentos - valores que a função usa quando é chamada
- Após a execução da tarefa pode retornar o resultado



Tipos de Funções

- Native:
 - Funções que fazem parte do motor Javascript (built into)
 - Ex: Math.max(10, 12), alert("test"), Date(), etc.

- Custom:
 - Funções criadas por nós *to perform* determinada tarefa

```
function square(x) {  
    return x * x;  
}
```

Funções

Declaração da Função:

```
function square(x) {  
    return x * x;  
}
```

Nome

Parâmetros

Corpo

Invocar a Função:

```
square(3);
```

Argumentos

```
return x * x;
```

return – indica qual o valor devolvido pela função

Funções - variáveis como argumentos

- O valor de um argumento pode ser dado usando uma variável;
- O argumento será o valor guardado na variável no momento em que a função é invocada.

```
function somar(x, y) {  
  return x + y;  
}
```

```
> let a = 5;  
> let b = somar(a, 3);  
  
>> b = 8
```

Funções - expressões como argumentos

- O valor de um argumento pode ser dado usando uma expressão;
- O argumento será o valor da expressão no momento em que a função é invocada

```
> let a = 5;
> let b = somar(a - 2, 3);

>> b = 6
```

Funções - resultado de funções como argumentos

- O valor de um argumento pode ser o valor devolvido por uma função;
- As invocações usadas como argumentos são executadas primeiro, de modo que o valor devolvido possa ser utilizado como argumento

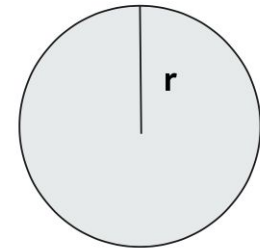
```
> let a = 5;
> let b = somar(subtrair(a, 2), 3);

>> b = 6
```

Exercício

1. Criar função que recebe 3 argumentos e devolve a soma dos dois primeiros, multiplicando depois o terceiro argumento;
2. Criar uma função que retorna a área de um círculo;
3. Criar função que converte a temperatura de graus Celsius para Fahrenheit;

$$\text{Área} = \pi r^2$$



$$F = \left[\frac{9}{5} C \right] + 32$$

Instrução if-else

- Permite condicionar a execução de uma ou várias instruções em função de determinada condição booleana (true/false)

```
if (condição) {
    executa código se condição é verdadeira
} else {
    executa código se condição é falsa
}
```

```
function min(a, b) {
    if (a < b) {
        return a;
    } else {
        return b;
    }
}
```

Instrução if-else

- Possível encadear várias condições para verificar várias condições:

```
if (condição_1) {
    executa código se condição_1 é verdadeira
} else if (condição_2) {
    executa código se condição_2 é verdadeira
}
```

Strings

- Em Javascript uma string é delimitada por aspas duplas (“ ”), simples (‘ ’) ou acentos graves (` `)

```
let string1 = “Ana”;  
let string2 = ‘Joana’;  
let string3 = `Mariana`;
```

Strings - propriedades e métodos

1. Tamanho da String

```
let string1 = "Ana";
> console.log(string1.length)
>> 3
```

2. Concatenação:

```
let string1 = "hello ";
let string2 = "world!";
string1.concat(string2);
>> "hello world!"

string1 + "everyone!";
>> "hello everyone!"
```

3. Comparação: (case-sensitive)

```
let string1 = "hello world";
let string2 = "HELLO WORLD";

string1 === string2;
>> false
```

Strings - propriedades e métodos

4. Interpolação

Permite incorporar valores ou expressões dentro de string. String tem se definida dentro de acentos graves (` `), para inserir valor de uma expressão:

```
let nome = "Ana";  
let idade = 30;  
let template = `A ${nome} tem ${idade} anos`;  
  
>> "A Ana tem 30 anos"
```

Exercício

1. Criar uma função que devolve:
 - Imprime “Está gelado”, se a temperatura for **inferior a 0**;
 - Imprime “Está frio”, se a temperatura for **inferior a 15**;
 - Imprime “Está quente”, se a temperatura for **superior a 50**;
 - Imprime “Está a ferver”, se a temperatura for **superior a 80**;
 - Imprime “Está normal”, para os outros casos.

Exercício

- Criar uma função que recebe o nome e a idade de uma pessoa e devolve uma String construída com uma mensagem que diz o ano que nasceu:
 - Ex: "Olá José, nasceste em 1993"

Strings (cont.)

- Podemos olhar para uma String como uma **sequência de caracteres**, e isso permite-nos obter o caracter específico numa certa posição:

```
let texto = `O José nasceu em 1993`;  
console.log(texto[2])  
  
>> "J"
```


Strings (cont.)

5. **Slice:** remove parte da string e devolve a parte extraída:

```
let texto = `Hello world!!`;
let result = texto.slice(0, 5);

>> "Hello"
```

start - primeira posição

end - posição onde
termina mas não
incluída no *split*

Strings (cont.)

6. **split**: divide a string em um array de substrings. Retorna o array criado e não altera a string inicial:

```
let texto = `Hello world!!`;
let result = texto.split(" ");

>> ["Hello", "world!!"]
```

separator - expressão
para efetuar o *split*

limit - limitar o
número de *splits*

Objetos

- Permite guardar diferentes informações e tipos de forma estruturada
- Os valores são associados a uma determinada chave/key

```
> console.log(objeto.nome)
>> "João"

> console.log(objeto.idade)
>> 30
```

Obter valor do campo

```
let objeto = {
  nome: "João",
  apelido: "Matos",
  idade: 30
};
```

Objetos

```
let objeto = {  
  nome: "João",  
  apelido: "Matos",  
  idade: 30  
}
```

```
// Adicionar um valor ao objeto  
objeto.altura = 190;
```

```
// Alterar um valor  
objeto.idade = 37;
```

```
// Adicionar, usando uma String como chave  
objeto['altura'] = 190;
```

```
// usar o valor de uma variável como chave  
let chave = idade;  
objeto[chave] = 37;
```

Objetos

```
let objeto = {  
  nome: "João",  
  apelido: "Matos",  
  idade: 30  
}  
  
// Obter chaves  
> console.log(Object.keys(objeto));  
>> ["nome", "apelido", "idade"]  
  
// Obter valores  
> console.log(Object.values(objeto));  
>> ["João", "Matos", 30]
```

```
// Remover chave / valor  
delete objeto.apelido;
```

Arrays

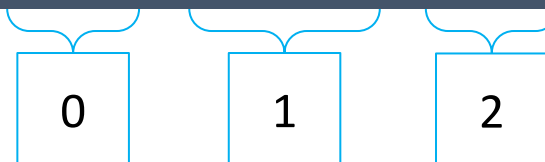
- Arrays (ou listas) permitem guardar uma quantidade indeterminada de informações em série;
- Podemos adicionar e aceder elementos através da posição em que eles estão

```
let carros = ["BMW", "Honda", "Audi"];
```

Arrays

- Aceder ao elemento através do índice:

```
let carros = ["BMW", "Honda", "Audi"];
```



```
console.log(carros[1]);
```

```
>> "Honda"
```

Arrays - tamanho

```
let carros = ["BMW", "Honda", "Audi"];
```

```
// obter tamanho do Array:
```

```
console.log(carros.length);
```

```
>> 3
```

```
let carros = ["BMW", "Honda", "Audi"];
```

```
// aceder ao último elemento do Array:
```

```
console.log(carros[carros.length - 1]);
```

```
>> "Audi"
```


Arrays - adicionar elemento

```
let carros = ["BMW", "Honda", "Audi"];

// adicionar um elemento:
carros.push("Mercedes");

>> ["BMW", "Honda", "Audi", "Mercedes"];
```

Arrays - encontrar index

```
let carros = ["BMW", "Honda", "Audi"];  
  
// encontrar posição/index do elemento:  
console.log(carros.indexOf("Honda"));  
  
>> 1
```

Arrays - método slice

```
let carros = ["BMW", "Honda", "Audi"];
```

```
console.log(carros.slice(1));
```

```
>> ["Honda", "Audi"]
```

Exercício

- Criar uma função que recebe um objeto com as propriedades **nome** e **anoNascimentos** e devolve o objeto com o **nome** e **idade**

```
let objeto = {  
  nome: "Francisco",  
  anoNascimento: 1998  
}  
  
function converterIdade(objeto){  
  ...  
}  
  
>> {  
  nome: "João",  
  idade: 26  
}
```

Exercício

Criar uma função que devolve um nome aleatório:

Dica: o método `Math.random()` devolve um valor aleatório de 0 a 1, nunca chegando a sair de 1; e `Math.floor()` retira as casas decimais de um número.

```
let nomes = ["João", "Maria", "Teresa"];
let sobrenomes = ["Silva", "Santos", "Carvalho"];

function nomeAleatorio(nomes, sobrenomes){
  ...
}

nomeAleatorio(nomes, sobrenomes);

>> "Teresa Silva"
```

Exercício

Criar uma função que devolve um nome aleatório:

Dica: o método `Math.random()` devolve um valor aleatório de 0 a 1, nunca chegando a sair de 1 ; e `Math.floor()` retira as casas decimais de um número.

```
function nomeAleatorio(nomes, sobrenomes){  
  console.log(`  
    ${nomes[Math.floor(Math.random() * 3)]} ${sobrenomes[Math.floor(Math.random() * 3)]}  
  `)  
}
```

- Para implementar um código flexível que consiga aceder aos elementos de um Array requer a utilização de **ciclos**. Podemos usar o mesmo código repetidamente para diferentes valores
- Como vimos, Arrays podem conter dimensões variadas.
- `let nomes = ["João", "Maria", "Teresa"];`

```
console.log(nomes[0]);  
>> "João"  
  
console.log(nomes[1]);  
>> "Maria"  
  
console.log(nomes[2]);  
>> "Teresa"
```

```
for (let i = 0; i < nomes.length; i++) {  
    console.log(nomes[i]);  
}  
  
>> "João"  
>> "Maria"  
>> "Teresa"
```

Ciclo *While*

- Permite correr o mesmo bloco várias vezes, enquanto uma determinada condição se mantiver;
- Código dentro do ciclo *while*, é corrido enquanto o valor na variável “contador” for superior a zero.

```
let contador = 5;
while (contador > 0) {
  console.log(contador);
  contador = contador - 1;
}

>> 5
>> 4
>> 3
>> 2
>> 1
```


Ciclo *While*

- Podemos indicar uma condição que é sempre verdadeira (“true”) e para o ciclo com uma condição interna através da instrução “**break**”;

```
let contador = 5;
while (true) {
  console.log(contador);
  contador = contador - 1;
  if (contador === 0)
    break;
}

>> 5
>> 4
>> 3
>> 2
>> 1
```

Ciclo *For*

- Sintaxe alternativa ao **while**, mais organizada para iterações em Arrays.

```
for (instrução_1; instrução_2; instrução_3)
```

- `instrução_1`: executa uma vez antes do bloco de código;
- `instrução_2`: condição a verificar para executar o bloco de código;
- `instrução_3`: executada sempre depois do bloco de código.

```
for (let contador = 5; contador > 0; contador--) {  
    console.log(contador);  
}
```

```
>> 5           >> 4  
>> 3           >> 2  
>> 1
```

Ciclo *For*

Exemplos:

```
for (let i = 0; i < nomes.length; i++) {  
    console.log(nomes[i]);  
}
```

```
>> "João"  
>> "Maria"  
>> "Teresa"
```

```
for (let i = nomes.length - 1; i >= 0; i--) {  
    console.log(nomes[i]);  
}
```

```
>> "Teresa"  
>> "Maria"  
>> "João"
```

Exercício

- Imprimir na consola todos os números pares de 1 a 100, usando as duas estruturas de ciclos (**for** e **while**)

Nota: operador '%' retorna o resto de uma divisão.

Array de Objetos

- Um array pode ter elementos de tipos diferentes:

```
let array = ["João", 1, true, {}];
```

- Array de objetos é uma estrutura de dados que armazena uma coleção de objetos:

```
let array = [
  {
    nome: "João",
    idade: 20
  },
  {
    nome: "Teresa",
    idade: 23
  }
];
```

Exercícios

- Imprimir o nome dos alunos do array de objetos:

```
let alunos = [  
  {  
    nome: "João",  
    idade: 20  
  },  
  {  
    nome: "Teresa",  
    idade: 23  
  },  
  {  
    nome: "Maria",  
    idade: 25  
  }  
];
```

Funções como argumento

- Funções de **callback** operam como variáveis normais enviadas no argumento de outra função, e são invocadas por essa função:

```
function teste1(){
    return "Bom dia!";
}
imprimir(teste1);
```

```
function imprimir(funcao){
    console.log(funcao());
}
```

```
function teste2(){
    return "Hello World!";
}
imprimir(teste2);
```

Funções como argumento

```
function imprimirNvezes(n, funcao) {  
  for (let i = 0; i < n; i++) {  
    console.log(funcao(i));  
  }  
}
```

```
function teste1(num) {  
  return 2 ** num;  
}  
imprimirNvezes(10, teste1);
```

```
function teste2(num) {  
  return 3 ** num;  
}  
imprimirNvezes(10, teste2);
```


Método .forEach()

- Chama a função para cada elemento do array;
- Coloca o valor do elemento do array no primeiro parâmetro dessa função;
- Função executada é chamada de “callback”

```
let alunos = [
  {nome: "João", idade: 20},
  {nome: "Teresa", idade: 23},
  {nome: "Maria", idade: 25}
];

function imprimirNome(aluno){
  console.log(aluno.nome);
}

alunos.forEach(imprimirNome);
```

Funções anónimas

- Nem sempre é necessário declarar uma função isoladamente para o código a conseguir chamar.
- Uma função anônima é uma função que não tem nome, e pode ser declarada diretamente no *callback* do método ***forEach***

```
let alunos = [  
  {nome: "João", idade: 20},  
  {nome: "Teresa", idade: 23},  
  {nome: "Maria", idade: 25}  
];  
  
alunos.forEach(function (aluno) {  
  console.log(aluno.nome);  
});
```

Exercício

1. Cria um algoritmo que imprime o nome e a idade de cada aluno do array de objetos, utilizando o método **.forEach()**
2. Imprimir apenas os alunos que têm mais de 21 anos

```
let alunos = [  
  {  
    nome: "João",  
    idade: 20  
  },  
  {  
    nome: "Teresa",  
    idade: 23  
  },  
  {  
    nome: "Maria",  
    idade: 25  
  }  
];
```

Método .map()

- O método **.map()** itera um array e devolve uma lista nova, com o resultado devolvido pelo callback;
- Não altera o array inicial;

```
let alunos = [  
  {nome: "João", idade: 20},  
  {nome: "Teresa", idade: 23},  
  {nome: "Maria", idade: 25}  
];  
  
let nomes = alunos.map(function (aluno) {  
  return aluno.nome;  
});  
  
console.log(nomes);  
  
>> ["João", "Teresa", "Maria"]
```

Método .filter()

- O método **.filter()** devolve um array novo, com os elementos do array inicial que passaram numa função de validação;
- Não altera o array inicial;

```
let alunos = [  
  {nome: "João", idade: 20},  
  {nome: "Teresa", idade: 23},  
  {nome: "Maria", idade: 25}  
];  
  
let alunos_21 = alunos.filter(function (aluno) {  
  return aluno.idade > 21;  
});  
  
console.log(alunos_21);  
  
>> [{nome: "Teresa", idade: 23},  
  {nome: "Maria", idade: 25}]
```

Exercício

1. Escreve um algoritmo que transforma a lista à direita numa lista de nomes completos dos alunos utilizando a função map:

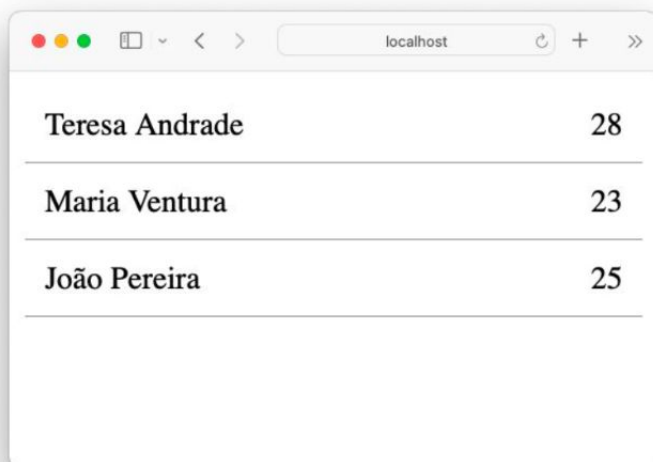
```
["João Andrade", "Teresa Ventura", "Maria  
Pereira"];
```

2. Usando o método filter(), imprime o array acima mas apenas os alunos com mais de 21 anos.

```
let alunos = [  
  {  
    nome: "João",  
    apelido: "Andrade",  
    idade: 20  
  },  
  {  
    nome: "Teresa",  
    apelido: "Ventura",  
    idade: 23  
  },  
  {  
    nome: "Maria",  
    apelido: "Pereira",  
    idade: 25  
  }  
];
```

Exercício

- Utilizando a seguinte lista de alunos, adicionar ao elemento div os dados do objeto alunos e apresentar da seguinte forma:



A screenshot of a web browser window displaying a table with three rows of student data. The browser's address bar shows 'localhost'. The table has two columns: the first column contains the full names of the students, and the second column contains their ages. The rows are: Teresa Andrade (28), Maria Ventura (23), and João Pereira (25).

Teresa Andrade	28
Maria Ventura	23
João Pereira	25

```
let alunos = [  
  {  
    nome: "Teresa",  
    apelido: "Andrade",  
    idade: 28  
  },  
  {  
    nome: "Maria",  
    apelido: "Ventura",  
    idade: 23  
  },  
  {  
    nome: "João",  
    apelido: "Pereira",  
    idade: 25  
  }  
];
```

```
<div class="lista_alunos"></div>
```